

Subgraphs & Workflow Patterns

LangGraph Session Notes — Saturday

Topics: Subgraphs · Send API · 6 Workflow Patterns · Orchestrator-Worker · Evaluator-Optimizer

❖ 1. Subgraphs — Graph Inside a Graph

A **subgraph** is a complete LangGraph graph used as a **node inside a parent graph**.

It has its own state, nodes, and edges — fully independent.

Why use subgraphs:

- Break complex workflows into smaller, reusable, testable pieces
- Each subgraph can be developed and tested independently
- Parent graph stays clean — complexity is hidden inside subgraphs

Two ways to use a subgraph in a parent graph

Shared State Keys

Subgraph shares at least one state key with parent.

Add directly as a node — no wrapper needed.

```
builder.add_node(  
    "subgraph",  
    subgraph.compile())
```

LangGraph handles state passing automatically.

Different Schema

Subgraph has a completely different state structure.

Need a wrapper function to transform state.

```
def wrapper(state):  
    sub_state = transform(state)  
    result = subgraph.invoke(sub_state)  
    return transform_back(result)
```

You manually convert between schemas.

❖ 2. Send API — Dynamic Parallel Execution

Problem with regular edges: Static — defined once at compile time. Cannot create dynamic parallel branches.

Send solves this: Dispatches tasks to nodes/subgraphs in parallel at runtime. You don't need to know how many branches at compile time.

```
from langgraph.types import Send  
  
def orchestrator(state):  
    topics = state["topics"] # e.g. ["RAG", "Agent", "Fine-tuning"]  
    return [Send("research_worker", {"topic": t})  
            for t in topics]
```

This sends each topic to the `research_worker` node in parallel.

3 topics = 3 parallel worker executions simultaneously.

Key difference — Send vs regular edge:

Regular edge: $A \rightarrow B$ (always goes to B, one at a time)

Send: $A \rightarrow [B, B, B]$ (multiple parallel B instances)

Fan-out = Send dispatches to multiple workers in parallel

Fan-in = results collected back into parent state using reducer

Reducer needed for fan-in — use `Annotated[list, operator.add]` to merge results:

`results: Annotated[list, operator.add]`

❖ 3. Six Workflow Patterns in LangGraph

Pattern 1 — Sequential

Each node runs one after another in a fixed order. Deterministic — always the same path.

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow \text{END}$

Use when: each step depends on the previous step's output.

Pattern 2 — Conditional Routing

Agent decides which path to take based on output. Uses conditional edges.

$A \rightarrow \text{decision_fn} \rightarrow B \text{ (or C or D)}$

Use when: different inputs need different processing paths.

Pattern 3 — Fan-out / Fan-in (Parallel)

One node dispatches to multiple workers in parallel using Send. Results collected back.

Orchestrator → Send → [Worker1, Worker2, Worker3]

↓ fan-in

Combine Results → END

Use when: independent subtasks can run simultaneously to save time.

Pattern 4 — Orchestrator-Worker with Send API

Orchestrator plans the work, Send dispatches dynamically to workers, results combined.

Your mentor's example: research topics [RAG, Agent, Fine-tuning] → 3 parallel research workers.

The number of workers is dynamic — decided at runtime by the orchestrator.

Pattern 5 — Evaluator-Optimizer Loop

Agent generates output → evaluator checks quality → if not good enough, loops back to regenerate.

Generator → Evaluator → good? → END

→ not good? → Generator (retry)

Has a maximum attempt limit (e.g. 3 tries) to prevent infinite loops.

Use when: output quality matters and needs iterative refinement.

Pattern 6 — Tool-Calling Agent Loop (ReAct)

LLM decides to call a tool → tool executes → LLM processes result → loops until final answer.

LLM → call tool? → ToolNode → LLM → done? → END

This is the standard ReAct pattern you built in Challenge 3.

❖ 4. LangGraph APIs Learned So Far

StateGraph — creates the graph with a defined state schema

add_node — adds a function or subgraph as a node

add_edge — sequential connection between nodes

add_conditional_edges — routing based on output

compile() — builds the executable graph

interrupt() — pauses graph for human input

Command — resumes paused graph with human response

Send — dispatches parallel tasks dynamically at runtime

★ *Rule: More APIs = more control. Learn when to use each one, not just how.*

❖ 5. Quick Reference

Subgraph = full graph used as a node inside parent graph

Shared state = add subgraph directly with `add_node`

Different schema = wrap subgraph in a function to transform state

Send = dispatch tasks in parallel at runtime

Fan-out = one node triggers multiple parallel workers

Fan-in = parallel results merged back using reducer

Reducer = `Annotated[list, operator.add]` — merges results from parallel nodes

Sequential = fixed order, deterministic

Conditional = branching based on output

Parallel = multiple nodes run simultaneously using `Send`

Evaluator loop = generate → evaluate → retry (with max attempts)

ReAct loop = LLM → tool → observe → loop until done

Coming next: Multi-agent flows, MCP, evaluation, project